Entity / Entity Value

An object that exist in the real world.

- virtual — Eg. order
- physical — Eg. product

Attributes

Prop. that define entity.
(Behaviours)

Entity Type

Collection of entities.

Q Define Entity, Entity Type & Attributes for :-

One can buy 1 kg onion @ ₹30 and 1 kg potato @ ₹40.
We have 20 kg onion & 50 kg potato available at big Basket store.

available at ✓ big Basket store ?

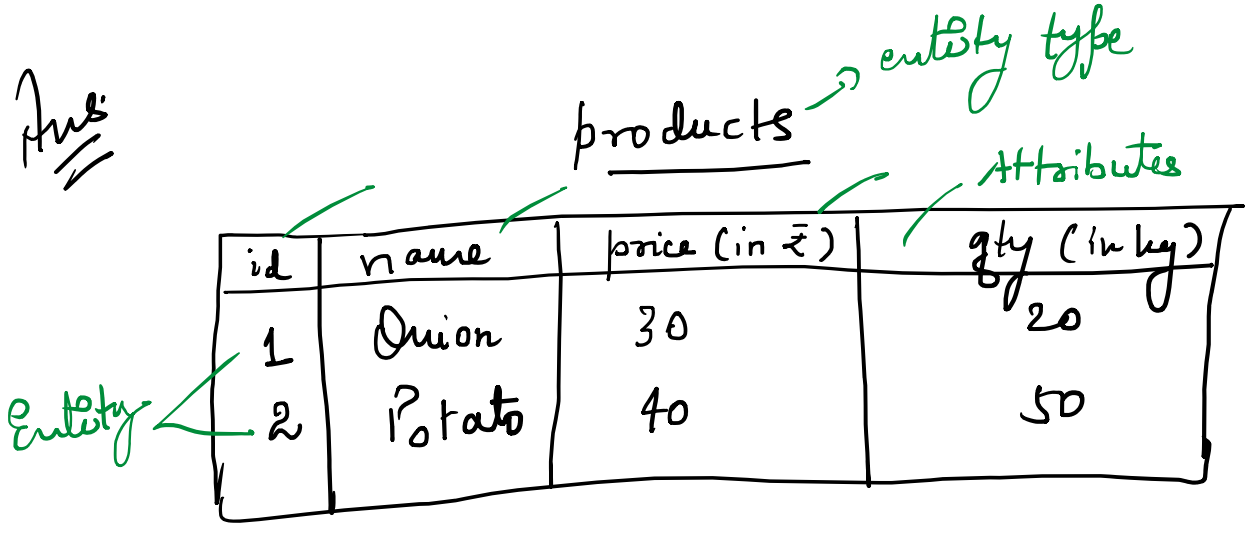
Ans Entity (value) ⟶ Onion & Potato.

Entity Type ⟶ Vegetables

Attributes ⟶ Price & Stock

Q Represent above ques data in form of a table & model them into entities /ET as described above.
What would be rows & col's ?

Ans



⊛ Types of Attributes.

1.) Simple :- can exist independently, atomic

1.) Simple :- can exist independently, attribute

Eg. name,

not too much relationship on top of it

2.) Composite :- composed internally of multiple attributes.

Eg. Address → state
→ zip code
→ landmark
→ city
⋮

(can be broken down)

3.) Single - Valued Eg. → Roll No.; ...

4.) Multi-valued Eg. → emails
subjects
phoneNo.

5.) Derived Attributes → Eg. Age → derived from DoB

↓ goes to DB

dynamic values
(keeps changing regularly)

because

because otherwise we would have to have a recurring job/ cronjob to update eligible records every 24 hrs.

goes to DB

if really reqd, own the dynamic update as well

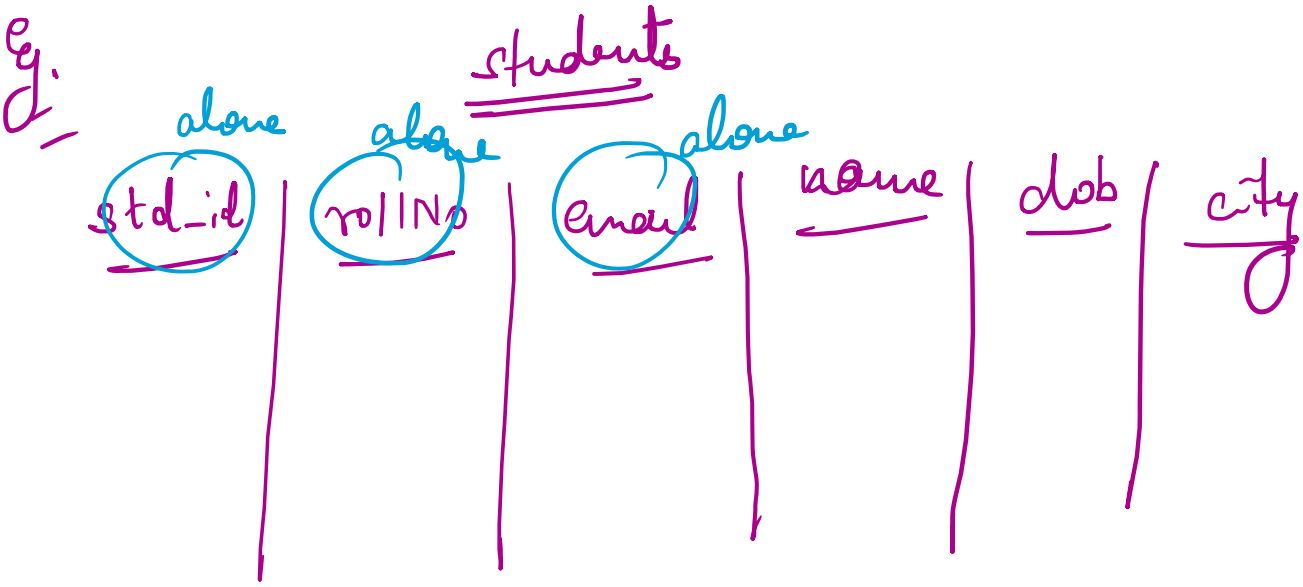
don't store at all

6.) Key Attribute (aka KEYS) → Attributes / grp. of attr. that help uniquely identify an entity.

7.) Candidate key → Min. key that helps to uniquely identity an entity.

Min. ≠> Single.

Eg.



key → std_id + rollNo. → uniquely identifies
std_id + rollNo.

CK → std_id
std_id + rollNo + email ✗. (Composite key)
~~name~~ ✗

Candi/key → roll No. ✓
std_id. ✓
~~rollNo + std_id~~ ✗
email ✓
~~rollNo + email~~ ✗
~~email + name~~ ✗.

~~email + name~~ ✗

Key → rollNo + name ✓

rollNo ✓

rollNo + name + email ✓

rollNo + email ✓

~~name + dob~~ ✗

std_id + rollNo + email + name + dob + city ✓

⊛ Primary Key

- any one CK
  - (Unique)
  - Not NULL

only 1 PK/table

Note

Practically, PK → never string. (always be a integer)
- indexing becomes latent (slow)
- infact never an

↳ infact, never an attribute that cx can control — changes.

↓

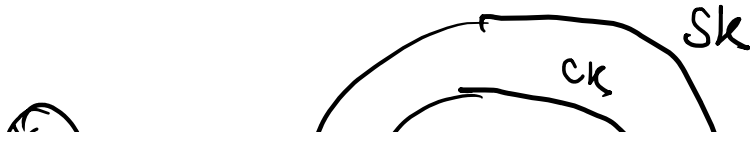
indexing then requires complete re-arrangement w/ every update

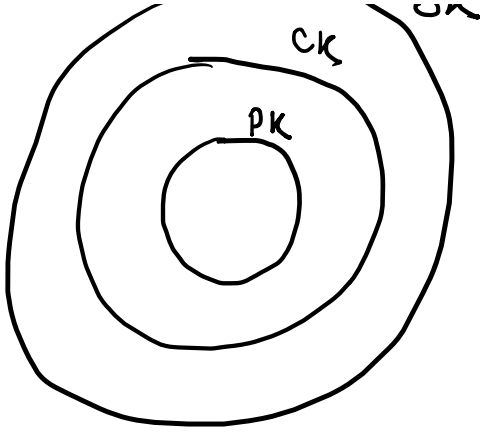
generally → pk → uuid / GUIDs

in distributed sys::

⊛ Super Key (kind of synonymous w/ key)

eg. rollNo + name ✓





Note

PK

↳ Internally → 1.) DB sorts data by PK (clustered index)

2.) DB o/p's every result, by default, based on PK

3.) DB creates index on PK

↳ A good PK

↳ 1.) Be FAST to sort on

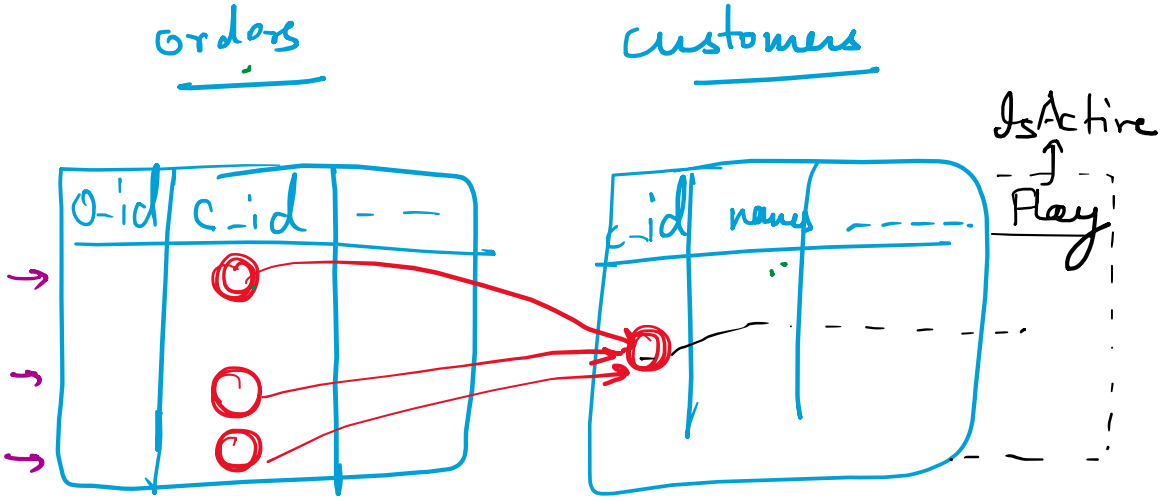
↳ 2.) Have smaller size

↳ 3.) Not get changed.

Note :-

ck can be a composite key.

(*) Foreign Keys → Help to uniquely identify rows in other referenced table



Q Can I delete an entry in the foreign table (Customer table) / Referenced table ?

(remove ref.)

Option 1

Deletes

Soft Delete

Data remains marked obsolete

Hard Delete (customers)

May be the company doesn't want to remove orders data

↓

Sol^{n}

↓ ensure

Write queries accordingly to filter for

↙ ↘

hard-deleted data

soft-deleted data

↓

↓

Outer Joins

conditions added
(Is Active flags)